Spend Pixels Where It Hurts: Hazard-Aware Rendering for Optical Long-Context Compression

Anonymous CVPR submission

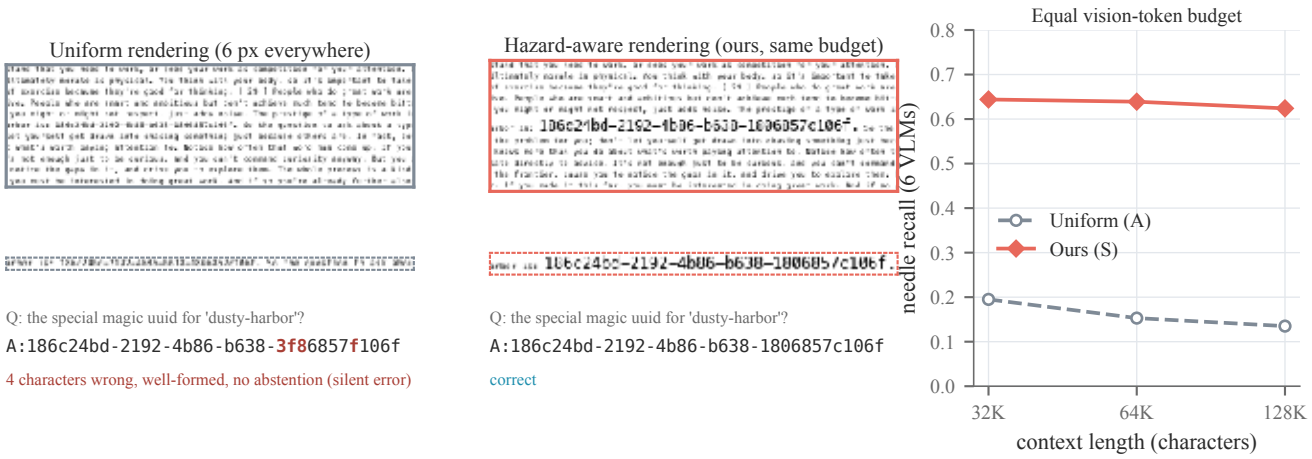


Figure 1. **Uniform rendering silently corrupts exactly the strings that must be copied.** A 32K-character essay hides one UUID. *Left:* rendered uniformly at 6 px, Claude Sonnet 5.5 returns a well-formed UUID with four wrong characters and no sign of doubt. *Middle:* our renderer enlarges the UUID and shrinks the surrounding prose; the image uses no more vision tokens, and the answer is exact. *Right:* needle recall averaged over six VLMs and three RULER tasks; uniform rendering degrades with context length while hazard-aware rendering does not.

Abstract

Rendering long text into images lets vision-language models (VLMs) read it with several times fewer tokens than the text itself. This optical compression spends resolution uniformly, yet the errors it causes concentrate in a few high-entropy strings such as identifiers and hashes. These strings are misread into well-formed but wrong answers, and more often as contexts grow. Our method rests on one fact: the renderer, unlike the model, sees the text. A training-free renderer for black-box VLMs flags verbatim-hazardous strings and enlarges only them within the vision-token budget of uniform rendering. It spends the budget this saves on verbatim anchors of the flagged strings, and it routes documents too dense in such strings back to plain text. Across six commercial and open VLMs, the allocation alone raises needle recall on RULER over uniform rendering by 0.45–0.49 at 32K–128K characters while using 15–28% fewer billed tokens. It beats font/size search and render-then-downsample in 63 of 72 paired comparisons and loses none, whereas enlarging random words does not help. With anchors, recall of identifiers that occur naturally in LongBench documents rises from 0.23 to 0.76 at about one fifth of the plain-text cost, synthetic needles in sparse documents reach 0.99, and LongBench QA shows no significant loss in any of 12 reader–task cells. Optical compression thus becomes usable for long documents that contain identifiers, with a text-side rule that decides when not to use it at all.

1. Introduction

Long contexts are expensive to feed to large models. A growing line of work reduces this cost by *rendering* text into images and letting a vision-language model (VLM) read the pixels [7, 29, 36]. A page of small glyphs costs a fixed number of vision tokens regardless of how many characters it holds, so dense rendering can compress a document several-fold. In our measurements, a 128K-character essay costs 27.1K billed text tokens but only 5.0K billed tokens as a uniformly rendered image. The appeal is practical: the compression needs no training, works with any VLM that

accepts images, and is controlled entirely by the party that renders the text.

Existing systems decide *how densely* to render, but not *where* the resolution goes. Optical compressors choose one font, size, and layout for the whole document, sometimes searched for a good accuracy–compression trade-off [7]. Training-free visual-token reduction methods prune or merge tokens after the image is encoded, using visual saliency or attention [5, 41, 43]. Text-side compressors drop tokens that a language model judges unimportant [15, 21]. None of these signals captures a property that many downstream uses depend on: whether a span must be reproduced *character for character*.

This blind spot is costly because verbatim risk is not uniform. Identifiers, hashes, file paths, and long numbers carry no redundancy, so a single misread glyph turns them into a different, equally plausible string, while the surrounding prose survives the same blur. Fig. 1 shows the typical outcome: a VLM returns a well-formed UUID with four wrong characters and no hedge. We find three properties of this failure that existing allocation schemes do not address. First, it is *concentrated*: at an equal budget, uniform rendering recovers only 9–28% of RULER needles (Tab. 1), whereas short prose facts rendered at the same densities are recovered far more reliably (Sec. C). Second, it is *silent*: under uniform rendering, about one in eight needles is answered with a well-formed near-miss of the correct string. In our test, a reader explicitly allowed to answer "[unreadable]" used that option for at most 6% of identifiers, so a downstream system cannot tell such answers from correct ones. Third, it *worsens with length*: averaged over six VLMs, needle recall under uniform rendering falls from 0.20 at 32K characters to 0.13 at 128K. Vision-side token reduction is not designed for this: once text is rasterized, which glyphs form a hash is no longer explicit. Text-side compression is not designed for it either, because it removes text rather than protecting it.

Our observation is that the renderer already holds the missing signal. Before any pixel is drawn, the text says where the hazardous strings are, and the renderer decides how many pixels each of them gets. We turn this into *hazard-aware rendering* (HAR, Fig. 2), four training-free steps that require only black-box access to the VLM: DETECT flags verbatim-hazardous strings with label-free rules; ALLOCATE enlarges only those strings and shrinks prose so that the image stays within the vision-token budget of uniform rendering; ANCHOR spends the budget this saves on *anchors*, verbatim text copies of the flagged strings; and ROUTE sends documents whose anchors would be too expensive back to plain text, using a ratio computable before any model call. In our experiments, S denotes the allocation step alone and X allocation plus anchors.

Our contributions are:

- We characterize verbatim corruption under optical compression across six VLMs, showing that it is concentrated on high-entropy strings, silent, and growing with context length (Sec. 5).
- We propose budget-constrained, content-aware glyph allocation for rendered context. At equal or lower cost, it raises needle recall by 0.42–0.57 over uniform rendering at 16K–128K characters. It is significantly better than a font/size search or render-then-downsample baseline in 63 of 72 paired comparisons and worse in none, and an ablation that enlarges random words instead shows that the gain comes from targeting the hazardous strings.
- We show that reallocation and anchors are complementary. At a 6 px budget, the vision tokens saved by reallocation pay for the anchors. Together they more than triple the recall of identifiers occurring naturally in LongBench documents (0.23 to 0.76) at $\approx 0.2\times$ the plain-text cost, without significant loss on LongBench QA. With a density-aware router, they dominate anchors on uniform images at every operating point.
- We map where the approach applies: a detector ablation shows that when hazards go undetected the rendering falls back to uniform, and a profile of real corpora shows that prose-dominated documents are routed to images while code and logs are routed to text.

2. Related Work

Optical context compression. Reading text from pixels has a long history in representation learning, from pixel-based language models [11, 29] to unified image–text encoders [32] and screenshot pretraining [18]. Recent systems use rendering explicitly to compress long contexts: DeepSeek-OCR decodes text from a small number of vision tokens [36, 37], Glyph renders long inputs and searches for a good global rendering configuration [7], and other work renders distant context or interleaves rendered text with images [23, 34, 39]. Evaluations have started to show where this fails: compressed reading loses retrieval ability [45], accuracy collapses when linguistic priors are removed [12, 24], the text–pixel gap depends strongly on font and resolution [31], and optical compression can underperform simple learned compressors [17]. The closest methods vary resolution within the input, but they are trained: FocusVTC combines a low-resolution global view with adaptively enhanced regions [46], and SEER learns query-dependent selective compression [40]. We share the premise of these systems but change *where* the pixels go, with a rule that is training-free, query-independent, and applicable to any black-box VLM. Because the rule does not depend on the question, a rendered document can be cached and reused across queries. The failures documented above concern the same kind of high-entropy, prior-free strings that our allocation protects.

Training-free visual token reduction. A complementary line of work reduces the tokens of an already encoded image: FastV drops low-attention tokens after an early layer [5], SparseVLM uses text guidance [43], VisionZip and PruMerge keep dominant or merged tokens [30, 41], ToMe merges similar tokens [4], and recent methods balance importance and diversity or target the output objective [10, 26, 42]. RTPrune adapts pruning to DeepSeek-OCR by exploiting text density [33], while FCoT-VL shows that text-rich images need dedicated compression [20]. These methods operate inside the model on visual features, where it is no longer explicit that a group of glyphs forms a hash. Our method acts before encoding, from the text, and is orthogonal: any of them could be applied to images produced by our renderer.

Text-side and memory compression. Prompt compressors drop tokens judged uninformative by a small language model [15, 16, 21], learned compressors map context to a few soft tokens [8, 13, 28], and KV-cache methods evict entries during inference [9, 22, 38, 44]. Selective Context, like our detector, uses an information-theoretic signal, but uses it to *remove* low-information text. Verbatim strings are high-information and must not be removed; we keep every character and only change how large it is drawn.

Evaluation and input resolution. We measure verbatim fidelity with RULER needle tasks [14], whose UUID and number values are exactly the strings at risk, and measure collateral damage on LongBench QA [3]; text-rich and long-document benchmarks [25, 27] document the same weakness in natural images. Modern VLMs accept dynamic resolutions and bill vision tokens roughly in proportion to pixel area [1, 2, 6, 19, 35], which makes the rendering budget a meaningful and controllable cost. We therefore report fidelity on synthetic needles and on naturally occurring identifiers separately from collateral damage, and we measure cost in provider-billed tokens rather than in pixels.

3. Method

Overview. Given a long document, an optical compressor renders it into image pages that a VLM reads in place of the text. Our goal is to make the strings that must be copied exactly survive this rendering, without spending more vision tokens and without access to the model. Figure 2 shows the four steps. DETECT reads the text and marks verbatim-hazardous strings. ALLOCATE renders those strings large and the remaining prose small, choosing sizes so that the image costs no more than uniform rendering would. ANCHOR appends *anchors*, verbatim text copies of the flagged strings, paid for by the vision tokens the allocation saved. ROUTE estimates, before calling the model, whether the anchors are cheap enough, and otherwise falls back to plain text.

3.1. Problem formulation

A document $x = (w_1, \ldots, w_n)$ is a sequence of whitespace-separated words. A renderer R maps x and a size assignment $\sigma : \{1, \ldots, n\} \to \mathbb{N}$ (glyph height in pixels for every word) to an image $I = R(x; \sigma)$ of fixed width, which is split into pages for the VLM. To compare renderings, we need their cost in the unit providers bill. Following the patch-based accounting of dynamic-resolution VLMs [35], we define the vision-token cost of an image with pages of width w_p and height h_p as

$$v(I) = \sum_{p} \left\lceil \frac{w_p}{28} \right\rceil \left\lceil \frac{h_p}{28} \right\rceil , \tag{1}$$

where each vision token covers a 28×28-pixel area (14-pixel patches merged 2×2 in Qwen2-VL). Equation (1) is used only to enforce budgets; all costs we report are the prompt tokens billed by each provider. The *budget* is the cost of uniform rendering at a base size s_0, $B = v\big(R(x; \sigma \equiv s_0)\big)$, so every method we compare uses the same number of vision tokens as the uniform baseline, up to page rounding. Let $\mathcal{H}(x) \subseteq \{1, \ldots, n\}$ be the indices of hazardous words, those whose value is needed verbatim downstream. We seek a size assignment that makes every w_i, $i \in \mathcal{H}(x)$, as legible as possible subject to $v(I) \leq B$, without deleting or reordering any character.

3.2. DETECT: text-side hazard detection

The detector approximates $\mathcal{H}(x)$ without labels. Its premise is that a string is hazardous when it is *high-entropy*: a misread character yields another valid-looking string, and the language model cannot repair it from context. After stripping surrounding punctuation, we flag a word if it matches one of five patterns: a hexadecimal run of at least 8 characters, a run of at least 5 digits, a path containing a digit or a file extension, an IP address, or a digest prefix. We also flag words that mix letters and digits and either have at least 20 identifier characters or contain a 16-character window with Shannon entropy of at least 3.5 bits. Two choices encode what *not* to protect. Four-digit numbers are excluded because in natural text they are mostly years, which are low-entropy and cheap to read. The length and entropy rules require mixed letter–digit tokens so that long English words are not flagged. A preliminary detector without these two exclusions flagged on average 85–112 words per LongBench document, most of them years, and shrank prose in documents that did not need it (Sec. D).

When the detector flags nothing, the rendering is identical to the uniform baseline, so a document whose hazards all go undetected loses only the potential gain (Sec. 5.6).

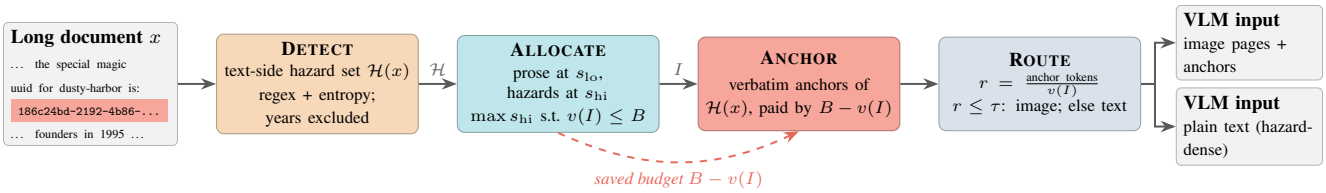


Figure 2. **Hazard-aware rendering.** DETECT marks verbatim-risky strings in the text before rendering; ALLOCATE enlarges only those strings and shrinks prose so that the image stays within the vision-token budget B of uniform rendering (up to page rounding) (a 6 px uniform image in our experiments); ANCHOR spends the budget the allocation saved on anchors, verbatim text copies of the flagged strings; ROUTE sends documents whose anchors would be too expensive to plain text. Every step is training-free and needs no access to the model.

A missed hazard in a document where *other* strings are flagged is, however, shrunk together with the prose.

3.3. ALLOCATE: budget-constrained glyph allocation

Given $\mathcal{H}(x)$, we restrict size assignments to two levels: prose words at s_{lo} and flagged words at s_{hi}. Two levels keep the layout simple; a locally varying alternative that shrinks only prose near flagged strings brought no further gain (Sec. D). Prose is wrapped at character boundaries like the uniform baseline, so it consumes no extra space, while each flagged word is kept on one line so it is never split. We choose the sizes by

$$(s^*_{\text{hi}}, s^*_{\text{lo}}) = \operatorname*{lexmax}_{\substack{s_{\text{hi}} \in [s_0,\, 12] \\ s_{\text{lo}} \in [5,\, \min(s_{\text{hi}}, s_0)]}} (s_{\text{hi}}, s_{\text{lo}}) \;\text{ s.t. }\; v\big(R(x; \sigma_{s_{\text{lo}}, s_{\text{hi}}})\big) \le B \tag{2}$$

where $\sigma_{s_{\text{lo}}, s_{\text{hi}}}$ assigns s_{hi} to words in $\mathcal{H}(x)$ and s_{lo} to all others, and lexmax first maximizes s_{hi} and then s_{lo}. The order expresses the design choice that the marginal pixel is worth more on a hash than on prose: verbatim accuracy is limited by the least legible character of a string, whereas prose comprehension degrades gracefully. The lower bound of 5 px keeps prose within the range VLMs still read, and $s_{\text{lo}} \le s_0$ ensures prose is never enlarged. If no pair satisfies the constraint, prose-only lines lose their single pixel of extra line spacing and the search is repeated; if $\mathcal{H}(x) = \emptyset$, the result is identical to uniform rendering. At $s_0 = 5$ px, prose is already at its minimum size, so this removed line spacing is what makes room for the enlarged strings; at $s_0 = 6$ px, prose shrinks to 5 px. With our unoptimized pure-Python renderer, budget computation plus allocation takes about 1 s for a 32K-character document and 4 s for 128K characters on one CPU core, small compared with a VLM call. Figure 1 (middle) shows a typical outcome: the UUID is drawn at 12 px while prose stays at 6 px.

3.4. ANCHOR: anchors paid by the saved budget

When prose can shrink ($s_0 = 6$ px), the allocated image usually ends *below* the budget, because the search stops at the largest feasible s_{hi} rather than filling every remaining row. In our long-context runs, it used 15–28% fewer billed tokens than uniform rendering; at $s_0 = 5$ px there is no such saving, and anchors add to the budget. We spend these savings on a second channel: after the question, we append the flagged strings verbatim as text, under a line stating that they are copied exactly from the image. Anchors do not say which key a string belongs to, so the reader must still locate it in the image. Enlarged strings should make this association easier, a plausible reason why anchors are more effective on the allocated image than on a uniform one at moderate density (Sec. 5.3).

3.5. ROUTE: density-aware fallback to text

Anchors are cheap only while hazards are sparse. Their text cost grows linearly with the number of flagged strings while the vision cost stays fixed, so before any model call we estimate

$$r(x) = \frac{\sum_{i \in \mathcal{H}(x)} c(w_i)}{v(I)}, \qquad c(w) = \tfrac{|w|}{2} + 3, \tag{3}$$

where $c(w)$ estimates the text tokens of one anchor (about two characters per token for identifiers plus three for the line break and separators) and I is the allocated image. If $r(x) \le \tau$ we send the image with anchors; otherwise we send the document as plain text, because a document this dense in identifiers is better served by its exact characters. We choose τ per reader by two-fold cross-validation on the density sweep (Sec. 5.3); the selected values have median 0.3, which we fix for a held-out reader (Sec. G) and for profiling real corpora (Sec. 5.6).

4. Experimental Setup

Readers. We treat every VLM as a black box and query it once per item, with no conversation history, at temperature 0. The six readers span vendors, scales, and open weights: Claude Sonnet 5.5, Gemini 3.5 Flash-Lite, GPT-6 Luna (minimal reasoning effort), Qwen3-VL-235B-A22B [1], Llama 4 Maverick, and MiniMax-M3. Costs are

the prompt tokens billed by each provider, which include the image, the question, and any appended copy; MiniMax does not report usage and is excluded from cost averages only. Empty responses and safety refusals (32 of 14,010 calls in the experiments reported here) are dropped; outputs that cannot be parsed, such as degenerate repetition, count as wrong.

Tasks. Verbatim fidelity is measured with RULER needle-in-a-haystack tasks [14], re-implemented with the official templates and Paul Graham essays as the haystack: *single* (one UUID value), *multi-key* (four keys, one queried, numeric values), and *multi-value* (four numeric values for one key). Contexts have 16K characters (Sec. 5.1), 32K–128K characters split into up to three pages (Sec. 5.2), or 32K characters with $k \in \{1, \ldots, 256\}$ UUID needles to vary hazard density (Sec. 5.3). Collateral damage on ordinary comprehension is measured on LongBench [3] MultiFieldQA-en and HotpotQA, using contexts of at most 40K characters so that one image holds a document; these samples were drawn after the detector was finalized. We use $N = 25$ items per task and reader for RULER at 16K and 32K characters, $N = 20$ for the 32K–128K length study, and $N = 50$ for LongBench.

Arms. All image arms share the renderer of Sec. 3, a 1008-pixel page width, and the budget B of uniform rendering at $s_0 \in \{5, 6\}$ px (≈21 and 14 characters per vision token). **A** is uniform rendering. **U** searches font family (monospace, Arial, Arial Narrow) and size for the largest uniform text that fits B, a training-free stand-in for global configuration search [7]. **Z** renders at 10 px and downsamples the image to B, the resolution-compression view of optical encoders [36]. **K** appends anchors, verbatim copies of the flagged strings, to A. **S** is our allocation (Sec. 3.3), **X** adds anchors to S (Sec. 3.4), and **T** gives the plain text. Prompts follow the task templates, with the context replaced by the image; RULER answers are requested as JSON.

Metrics and statistics. RULER is scored by recall of the gold values (string match, all values), LongBench by token-level F1, as in the original benchmarks. For every comparison we compute the mean paired difference over items with a 95% bootstrap confidence interval (10,000 resamples) and call a cell significant when the interval excludes zero; a *cell* is one reader on one task at one setting.

5. Results

5.1. Equal-budget comparison

Table 1 compares all image arms at the same vision-token budget on 16K-character RULER documents. Uniform rendering recovers only 9% of needles at the denser budget and 28% at the lighter one. The failures are spread across readers: at 6 px, Claude Sonnet 5.5 recovers 0.57 and Gemini 0.88 of the needles, while GPT, Qwen, Llama, and MiniMax recover at most 11%. Hazard-aware allocation (S) raises the mean to 0.66 and 0.70, a gain of 0.57 and 0.42, and is significantly better than uniform rendering in 33 of 36 cells. The two stronger uniform baselines barely help: searching font family and size (U) reaches 0.14 and 0.28, and rendering large then downsampling (Z) 0.17 and 0.23, plausibly because both spread legibility evenly over all characters. S is significantly worse than U or Z in none of the 72 paired comparisons, and at the lighter budget it is also 11% cheaper than uniform rendering, because prose shrinks more than the enlarged strings grow. The weaker readers benefit most in absolute terms: Llama 4 Maverick goes from 0.00 to 0.58 and MiniMax-M3 from 0.00 to 0.57 at 5 px.

Is it the targeting or the enlargement? Arm R runs the same allocation but enlarges randomly chosen ordinary words with the same total length as the flagged strings, leaving the needles at prose size. It recovers 0.11 and 0.09 of the needles, no better than uniform rendering and significantly worse in 5 of 15 cells at 6 px, where the needles shrink with the prose. S exceeds R by 0.53 and 0.56 (14 of 15 cells significant at each budget): the gain comes from *which* strings are enlarged, not from enlargement as such.

Anchors. Anchors on the uniform image (K) recover 0.97 and 0.95 of the needles for 5% more tokens than uniform rendering, a reminder that verbatim copies are hard to beat when hazards are few. Allocation leaves one failure mode untouched: the fraction of well-formed near-misses is similar for A and S (0.13 and 0.16 at 5 px), whereas anchors nearly remove it (0.002). Combining the two (X) gives the best of both. On the five readers on which both were run, X reaches 0.98 and 0.99 against 0.97 and 0.94 for K. At the 6 px budget, where allocation frees vision tokens, X is 0.06 more accurate than K (3 of 15 cells significant) and 9% cheaper (881 vs. 968 billed tokens); it is even 4% cheaper than uniform rendering alone. At the 5 px budget prose cannot shrink further, so X costs the same as K and matches its accuracy. We return to this trade-off as hazards become denser in Sec. 5.3.

5.2. Scaling with context length

Optical compression is most attractive for long documents, so Fig. 3 extends the comparison to 128K characters. Under uniform rendering, mean recall declines from 0.20 at 32K to 0.15 at 64K and 0.13 at 128K, consistent with prior reports that compressed reading loses retrieval ability as context grows [45]. Hazard-aware rendering is essentially flat at 0.64, 0.64, and 0.62, so its advantage widens slightly from

Table 1. **Equal-budget comparison on RULER needle retrieval (16K characters).** Recall averaged over three NIAH tasks (N=25 each) per reader, at two budgets (uniform 5 px ≈ 21 and 6 px ≈ 14 characters per vision token). A, U, Z, R, and S are images with the same vision-token budget: uniform, best uniform font/size, render-then-downsample, *random* words enlarged by our allocation, and hazardous strings enlarged (ours). K and X add anchors to A and S. **Bold**: best within each group. †R and X were run on the five readers other than Claude; their means and billed tokens are over those readers (on the same readers, K averages 0.97 and 0.94). Billed tokens average the readers that report usage. Hazard-aware allocation (S) is significantly worse than U or Z in none of 72 paired comparisons and better in 63; enlarging random words instead (R) does not help.

	5 px budget (≈21 chars/vtok)							6 px budget (≈14 chars/vtok)						
	equal-budget image					+ anchors		equal-budget image					+ anchors	
Reader	A	U	Z	R	S (ours)	K	X (ours)	A	U	Z	R	S (ours)	K	X (ours)
Claude Sonnet 5.5	0.14	0.22	0.19	–	**0.79**	**0.95**	–	0.57	0.64	0.46	–	**0.92**	**1.00**	–
Gemini 3.5 Flash-Lite	0.41	0.57	0.79	0.53	**1.00**	**1.00**	**1.00**	0.88	0.78	0.84	0.44	**1.00**	1.00	**1.00**
GPT-6 Luna	0.02	0.03	0.02	0.01	**0.50**	0.91	**0.94**	0.11	0.16	0.08	0.02	**0.65**	0.82	**0.98**
Qwen3-VL-235B	0.00	0.00	0.00	0.00	**0.55**	**1.00**	**1.00**	0.07	0.08	0.03	0.00	**0.60**	**1.00**	**1.00**
Llama 4 Maverick	0.00	0.00	0.00	0.00	**0.58**	**0.97**	**0.97**	0.00	0.01	0.00	0.00	**0.59**	0.90	**1.00**
MiniMax-M3	0.00	0.00	0.00	0.00	**0.57**	**1.00**	0.98	0.02	0.01	0.00	0.00	**0.42**	0.97	**1.00**
Mean	0.09	0.14	0.17	0.11†	**0.66**	0.97	**0.98**†	0.28	0.28	0.23	0.09†	**0.70**	0.95	**0.99**†
Billed tokens	779	775	768	808†	779	823	857†	913	891	887	901†	812	960	881†

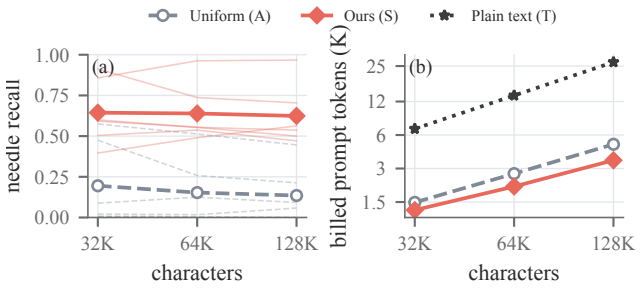


Figure 3. **Context length.** RULER at 32K–128K characters (6 px budget, up to three image pages, N=20 per task). (a) Needle recall; thin lines are individual readers, thick lines the mean over six readers. (b) Billed prompt tokens averaged over readers that report usage; note the log scale.

0.45 to 0.48 and 0.49, with 46 of 54 cells significant. The cost picture is equally favorable (Fig. 3b): reallocation uses 15%, 23%, and 28% fewer billed tokens than uniform rendering at the three lengths, and 5.4×, 6.6×, and 7.6× fewer than the plain text, which every reader we measured answers with recall of at least 0.99. Individual readers differ widely in level (0.47 to 0.97 at 128K for S) and in trend. S improves with length for Claude and MiniMax, is roughly stable for GPT, Qwen, and Llama, and declines for Gemini (0.91 to 0.70), whose uniform curve declines even more (0.48 to 0.21).

5.3. Hazard density, anchors, and routing

Figure 4 varies the number of hazardous strings per 32K-character document from 1 to 256. Reallocation alone degrades as density grows (0.50 at k=1, 0.33 at k=16, 0.13 at k=64), because more strings compete for the pixels prose can give up: the enlarged size falls from 12 px to 7 px at k=256. Anchors are strong while hazards are sparse but become both less accurate and expensive as they multiply, reaching 0.39 recall at k=64 for 1.8× the tokens of uniform rendering.

Combining the two (X) gives the best of both in the regime where images are worth using. At k=4, X recovers 0.99 of the needles while costing 13% *less* than uniform rendering and 20% less than K: the vision tokens saved by reallocation pay for the anchors. At k=16, X is 0.10 more accurate than K and 13% cheaper, because the reader must still find which copied string belongs to the queried key, and enlarged strings make that lookup easier. From k=64 on, no image arm recovers more than 0.39 of the needles, which marks where optical compression should not be used at all.

The router of Sec. 3.5 turns this into a policy. Sweeping its threshold traces a recall–cost frontier (Fig. 4c). On this frontier, routing between X and plain text dominates routing between K and plain text at every operating point: at a fixed τ=0.3, it reaches 0.97 recall at 0.69× the plain-text cost versus 0.93 at 0.70× (mean over the four readers that report billed costs for both images and text). On documents with at most 16 hazardous strings, X reaches 0.93 recall at 0.22× the plain-text cost, compared with 0.86 at 0.24× for K, and with $k \leq 4$ the routed system reaches 0.99–1.00 at 0.19–0.22×. With τ cross-validated per reader instead, the routed system reaches 0.99 recall at 0.71× the plain-text cost on held-out items; this aggregate reflects that half of

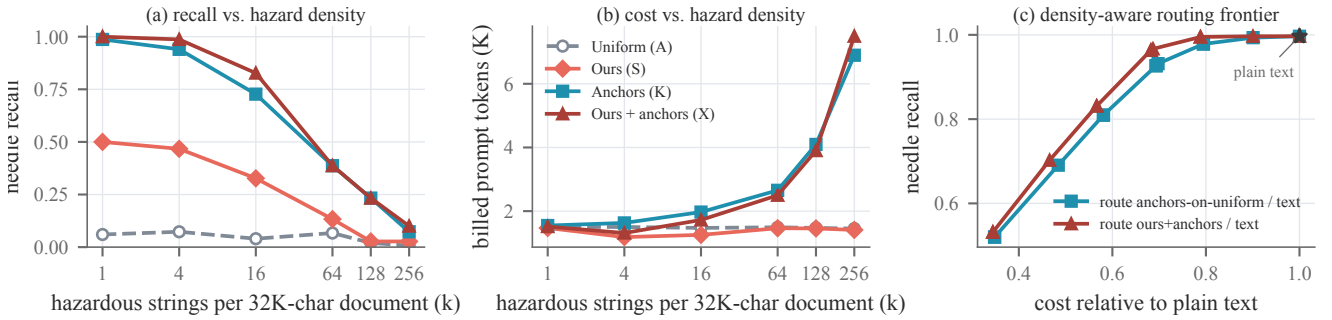


Figure 4. **Hazard density.** 32K-character documents with k UUID needles, one of which is queried (N=25). (a) Recall and (b) billed tokens for uniform rendering (A), our allocation (S), anchors on uniform rendering (K), and our allocation with anchors (X); recall averages six readers, cost the five that report usage (MiniMax does not). (c) Recall versus cost relative to plain text when each document is routed either to an image with anchors or to plain text by Eq. (3), sweeping τ (the four readers with billed costs for both images and plain text; Claude was not run on plain text at this length). Routing with X dominates routing with K at every threshold.

Table 2. **Naturally occurring identifiers.** Exact-copy recall on 60 LongBench documents ($\leq$40K characters, 6 px budget) for identifiers already present in the text, such as report numbers, ISBNs, grant numbers, citation keys, and URLs. *: paired 95% CI vs. uniform (A) above zero. Billed tokens average the four readers that report usage.

Reader	A	S	K	X	T (text)
Gemini 3.5 Flash-Lite	0.72	0.80	0.87	0.90*	0.92
GPT-6 Luna	0.25	0.55*	0.85	0.82*	1.00
Qwen3-VL-235B	0.12	0.48*	0.77	0.78*	0.98
Llama 4 Maverick	0.05	0.20*	0.55	0.62*	0.97
MiniMax-M3	0.03	0.39*	0.65	0.67*	0.98
Mean	0.23	0.48	0.74	0.76	0.97
Billed tokens	1181	1035	1267	1128	5324

Table 3. **Reallocation does not hurt comprehension.** LongBench QA F1 on held-out samples (N=50 per task, contexts $\leq$40K characters, 6 px budget). */†: paired 95% CI vs. uniform rendering (A) above/below zero. Ours + anchors (X) has no significantly negative cell.

	MultiFieldQA-en				HotpotQA			
Reader	A	S	K	X	A	S	K	X
Claude Sonnet 5.5	0.49	0.49	0.50	0.46	0.80	0.78	0.80	0.76
Gemini 3.5 Flash-Lite	0.47	0.47	0.53	0.47	0.59	0.62	0.58	0.57
GPT-6 Luna	0.36	0.39	0.37	0.36	0.68	0.73	0.68	0.70
Qwen3-VL-235B	0.27	0.26	0.31	0.26	0.66	0.61†	0.65	0.62
Llama 4 Maverick	0.28	0.25	0.34	0.29	0.45	0.50	0.51*	0.50
MiniMax-M3	0.23	0.26	0.25	0.25	0.29	0.34	0.41*	0.42*
Mean	0.35	0.35	0.38	0.35	0.58	0.60	0.61	0.60

the documents in this sweep are dense enough to be routed to text; real corpora are much sparser (Sec. 5.6).

5.4. Naturally occurring identifiers

RULER needles are synthetic, and our detector was designed with such strings in mind. To test identifiers that occur naturally, we built 60 questions from LongBench documents of up to 40K characters (ten each from Qasper, MultiFieldQA, GovReport, 2WikiMQA, HotpotQA, and MultiNews), each asking for the exact string that follows a unique six-word phrase. Targets are strings the documents already contain, such as report and grant numbers, ISBNs, citation keys, military specifications, dates, and URLs; we removed candidates produced by dataset artifacts such as words glued together across a missing space (Sec. E). Table 2 confirms the synthetic results on these strings. Allocation alone doubles mean recall (0.23 to 0.48, significant for four of five readers), and allocation with anchors more than triples it (0.76, significant for all five) while costing 11% fewer billed tokens than anchors on the uniform image and 5% fewer than uniform rendering. Plain text remains clearly better for the weaker readers (0.97 on average), at 4.7$\times$ the cost of X; only Gemini approaches it (0.90 vs. 0.92). Natural identifiers are thus where optical compression benefits most from protection, and where the router's option of falling back to text matters for weaker readers.

5.5. Effect on ordinary comprehension

Shrinking prose could hurt questions that have nothing to do with identifiers. Table 3 tests this on held-out LongBench documents of up to 40K characters, which contain natural identifiers, numbers, and formulas. Averaged over readers, F1 is unchanged on MultiFieldQA (0.35 for A, S, and X) and slightly higher on HotpotQA (0.58 for A, 0.60 for S and X). The full method X has no significantly negative cell among 12 and one significantly positive cell; reallocation alone has one significantly negative cell (Qwen3-VL on HotpotQA, -0.06). Anchors do not distract readers either: K is never significantly worse than A. Two design choices

Table 4. **Detector ablation.** RULER multi-value recall (16K characters, 6 px budget, N=25, mean of 5 readers; Claude was not run in this ablation). Seven-digit values are flagged by the regex detector; four-digit values are deliberately not (they collide with years), so the regex arms fall back to uniform rendering.

Arm	7-digit values (detected)	4-digit values (missed)
Uniform (A)	0.22	0.32
Ours, regex detector (S)	0.77	0.33
Ours, oracle detector	0.78	0.82
Ours + anchors, regex (X)	0.99	0.32
Ours + anchors, oracle	0.99	0.99

matter here. The detector excludes years and long English words (Sec. 3.2); the preliminary detector without these exclusions flagged most numbers in these documents and cost 0.05 F1 on average, with three significantly negative cells, on one LongBench sample, although it was neutral on another (Sec. D). And on almost half of the HotpotQA documents nothing is flagged, so the rendering is identical to uniform by construction.

5.6. Analysis

How much does the detector matter? Table 4 replaces the regex detector by an oracle that flags exactly the needle values. When the hazards are of a type the detector recognizes (seven-digit numbers), the regex detector is as good as the oracle, both for reallocation (0.77 vs. 0.78) and with anchors (0.99 vs. 0.99). When they are not (four-digit numbers, which the detector deliberately ignores), reallocation and anchors fall back to exactly the uniform result (0.33 and 0.32 vs. 0.32), while the oracle would reach 0.82 and 0.99. When a document's hazards all go undetected, the method thus costs the gain but does no harm. Hazards missed in documents where other strings are flagged would instead be shrunk with the prose, which this experiment does not test; improving detector recall, for example with task-specific patterns, is the remedy.

Where do real documents fall? The router decides from text alone, so we can profile real corpora without querying any model. We rendered 32K-character chunks from 13 LongBench tasks, essays, Python source, system logs, and package manifests (Sec. F). Prose-dominated documents (QA, summarization, essays) are routed to images 95–100% of the time at an estimated 0.19–0.27× the plain-text cost, and technical prose with occasional identifiers (MultiFieldQA) 75% of the time at 0.43×. Code is mostly routed to text (0.81–0.96×), and logs and manifests always are. This matches the density sweep: hazard-aware rendering is a tool for long documents that *contain* identifiers, not for documents that *consist* of them.

Which readers benefit? On short documents at four densities (Sec. C), the gain over uniform rendering follows an inverted U whose peak shifts with reader strength. The strongest readers gain most at the highest density (Claude +0.32 and Gemini +0.39 at 21 characters per vision token), where they first start to fail. Weaker readers gain most at moderate densities (+0.48 to +0.78 at 11–14). Reallocation helps wherever a reader is close to, but not past, its legibility limit, which is why a single budget benefits all six readers in Tab. 1. A seventh reader, Claude accessed through an agent interface, gives a consistent picture on a small sample (N=10 per cell, estimated costs): with τ=0.3 fixed from the other readers, it reaches 0.98 recall at about 0.61× the plain-text cost (Sec. G).

6. Limitations and Conclusion

Limitations. Most of our fidelity evaluation uses synthetic needles, and the natural-identifier set is small (60 automatically constructed questions). On that set, the full method still trails plain text by 0.21 recall on average, so weaker readers should be routed to text more often than our density threshold alone would suggest. We evaluate black-box readers only and therefore cannot say how enlarged glyphs change a model's internal attention. Budgets follow one patch-based accounting rule (Eq. (1)); providers tile images differently, and while billed costs confirm the savings for the readers we tested, other tokenizers may shift them. The detector is rule-based and recall-bounded: hazards it does not recognize receive no protection, and in documents where other strings are flagged they are shrunk with the prose. Finally, reallocation alone raises recall without reducing the share of confident near-misses; only anchors remove them.

Conclusion. Optical context compression decides how many pixels a document receives but not where they go, and its failures concentrate on exactly the strings that must survive verbatim. Because the renderer reads the text before it draws it, it can spend resolution where errors hurt, reinvest the savings in anchors, and fall back to text when a document is too dense with identifiers. These steps are training-free and model-agnostic. Across six VLMs, they make optical compression far more reliable for embedded identifiers, more than tripling their recall on natural documents at about one fifth of the plain-text cost. They also tell the system when a document is too dense with identifiers to be compressed at all.

References

[1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, et al. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025. 3, 4

[2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025. 3

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 3119–3137, 2024. 3, 5

[4] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *Proceedings of the International Conference on Learning Representations*, 2023. 3

[5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *Proceedings of the European Conference on Computer Vision*, pages 19–35, 2024. 2, 3

[6] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, et al. How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. *Science China Information Sciences*, 67(12):220101, 2024. 3

[7] Jiale Cheng, Yusen Liu, Xinyu Zhang, Yulin Fei, Wenyi Hong, Ruiliang Lyu, Weihan Wang, Zhe Su, Xiaotao Gu, Xiao Liu, et al. Glyph: Scaling context windows via visual-text compression. *arXiv preprint arXiv:2510.17800*, 2025. 1, 2, 5

[8] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3829–3846, 2023. 3

[9] Alessio Devoto, Maximilian Jeblick, and Simon Jégou. Expected attention: KV cache compression by estimating attention from future queries distribution. *arXiv preprint arXiv:2510.00636*, 2025. 3

[10] Zhengyao Fang, Pengyuan Lyu, Chengquan Zhang, Guangming Lu, Jun Yu, and Wenjie Pei. Prune redundancy, preserve essence: Vision token compression in VLMs via synergistic importance-diversity. In *Proceedings of the International Conference on Learning Representations*, 2026. 3

[11] Tianyu Gao, Zirui Wang, Adithya Bhaskar, and Danqi Chen. Improving language understanding from screenshots. *arXiv preprint arXiv:2402.14073*, 2024. 2

[12] Yonghan Gao, Zehong Chen, Lijian Xu, Jingzhi Chen, Jingwei Guan, and Xingyu Zeng. ZeroSense: How vision matters in long context compression. *arXiv preprint arXiv:2603.11846*, 2026. 2

[13] Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. In-context autoencoder for context compression in a large language model. In *Proceedings of the International Conference on Learning Representations*, 2024. 3

[14] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *Proceedings of the First Conference on Language Modeling*, 2024. 3, 5

[15] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376, 2023. 2, 3

[16] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 1658–1677, 2024. 3

[17] Ivan Yee Lee, Cheng Yang, and Taylor Berg-Kirkpatrick. Optical context compression is just (bad) autoencoding. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[18] Kenton Lee, Mandar Joshi, Iulia Raluca Turc, Hexiang Hu, Fangyu Liu, Julian Martin Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct: Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning*, pages 18893–18912, 2023. 2

[19] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. LLaVA-OneVision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025. 3

[20] Jianjian Li, Junquan Fan, Feng Tang, Gang Huang, Shitao Zhu, Songlin Liu, Nian Xie, Wulong Liu, and Yong Liao. FCoT-VL: Advancing text-oriented large vision-language models with efficient visual token compression. *arXiv preprint arXiv:2502.18512*, 2025. 3

[21] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6342–6353, 2023. 2, 3

[22] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems*, pages 22947–22970, 2024. 3

[23] Yanhong Li, Zixuan Lan, and Jiawei Zhou. Text or pixels? evaluating efficiency and understanding of LLMs with visual text inputs. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 10564–10578, 2025. 2

[24] Yunhao Liang, Ruixuan Ying, Bo Li, Hong Li, Kai Yan, Qingwen Li, Min Yang, Okamoto Satoshi, Zhe Cui, and Shiwen Ni. Visual merit or linguistic crutch? a close look at DeepSeek-OCR. *arXiv preprint arXiv:2601.03714*, 2026. 2

[25] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. OCRBench: On the hidden mystery of OCR in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024. 3

[26] Guangchuan Lv, Dianxing Shi, and Dingjie Fu. VPRune: Efficient training-free pre-LLM visual token pruning. *arXiv preprint arXiv:2609.24485*, 2026. 3

[27] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, et al. MMLongBench-Doc: Benchmarking long-context document understanding with visualizations. In *Advances in Neural Information Processing Systems*, pages 95963–96010, 2024. 3

[28] Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*, pages 19327–19352, 2023. 3

[29] Phillip Rust, Jonas F. Lotz, Emanuele Bugliarello, Elizabeth Salesky, Miryam de Lhoneux, and Desmond Elliott. Language modelling with pixels. In *Proceedings of the International Conference on Learning Representations*, 2023. 1, 2

[30] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. LLaVA-PruMerge: Adaptive token reduction for efficient large multimodal models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22857–22867, 2025. 3

[31] Kaiser Sun, Xiaochuang Yuan, Hongjun Liu, Chen Zhao, Cheng Zhang, Mark Dredze, and Fan Bai. Reading, not thinking: Understanding and bridging the modality gap when text becomes pixels in multimodal LLMs. *arXiv preprint arXiv:2603.09095*, 2026. 2

[32] Michael Tschannen, Basil Mustafa, and Neil Houlsby. CLIPPO: Image-and-language understanding from pixels only. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11006–11017, 2023. 2

[33] Ben Wan, Yan Feng, Zihan Tang, Weizhe Huang, Yuting Zeng, Jia Wang, and Tongxuan Liu. RTPrune: Reading-twice inspired token pruning for efficient DeepSeek-OCR inference. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 124471–124491, 2026. 3

[34] Alex Jinpeng Wang, Linjie Li, Yiqi Lin, Min Li, Lijuan Wang, and Mike Zheng Shou. Leveraging visual tokens for extended text contexts in multi-modal learning. In *Advances in Neural Information Processing Systems*, pages 14325–14348, 2024. 2

[35] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, et al. Qwen2-VL: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3

[36] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR: Contexts optical compression. *arXiv preprint arXiv:2510.18234*, 2025. 1, 2, 5

[37] Haoran Wei, Yaofeng Sun, and Yukun Li. DeepSeek-OCR 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026. 2

[38] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *Proceedings of the International Conference on Learning Representations*, 2024. 3

[39] Ling Xing, Alex Jinpeng Wang, Rui Yan, Xiangbo Shu, and Jinhui Tang. Vision-centric token compression in large language model. In *Advances in Neural Information Processing Systems*, pages 33080–33110, 2025. 2

[40] Jiawei Xu, Zhilin Zhai, Jinrui Fang, Ruohan Xu, Mingfei Lu, Yi Zhang, Guanchu Wang, Tianlong Chen, and Ying Ding. SEER: Long-context reasoning via selective visual-text compression. In *Proceedings of the Third Conference on Language Modeling*, 2026. 2

[41] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. VisionZip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 19792–19802, 2025. 2, 3

[42] Haokui Zhang, Congyang Ou, Dawei Yan, Peng Wang, Qingsen Yan, Yu Zhang, Ying Li, and Rong Xiao. TRIO: Token reduction via inference-objective guidance for efficient vision-language models. *arXiv preprint arXiv:2602.04657*, 2026. 3

[43] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis A. Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, and Shanghang Zhang. SparseVLM: Visual token sparsification for efficient vision-language model inference. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 74840–74857, 2025. 2, 3

[44] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, pages 34661–34710, 2023. 3

[45] Hongbo Zhao, Meng Wang, Fei Zhu, Wenzhuo Liu, Bolin Ni, Fanhu Zeng, Gaofeng Meng, and Zhaoxiang Zhang. VTCBench: Can vision-language models understand long context with vision-text compression? *arXiv preprint arXiv:2512.15649*, 2025. 2, 5

[46] FangZhi Zhong, Xuerui Qiu, Yuqi Pan, Ya Liu, Shaowei Gu, Bo Xu, and Guoqi Li. FocusVTC: Efficient and high-performance visual text compression with adaptive resolution. *arXiv preprint arXiv:2609.36651*, 2026. 2

LLM Usage Statement

Large language models play two roles in this work. First, they are the *objects of study*: the six vision-language models listed in Sec. 4 and the additional Claude reader of Sec. G were queried as black-box readers, and all reported numbers are computed by our scripts from their logged outputs. Second, the authors used an LLM-based coding assistant (Claude) to help implement the rendering and evaluation code, to generate figures and tables from the logged results,

and to draft and edit the text of this paper. The assistant also proposed experiments and analyses, which the authors reviewed and approved; the authors take responsibility for all claims. Every number in the paper is produced programmatically from the experiment logs, and every reference was checked against its primary source. No LLM was used to generate or label evaluation data beyond the readers' answers being evaluated.

A. Implementation Details

Renderer. Pages are 1008 pixels wide (36 patches) with a 4-pixel margin and are rendered in grayscale with the Menlo monospace font through PIL. Prose is wrapped at character boundaries; a flagged word is placed whole on the next line when it does not fit. Each line is as tall as its largest glyph plus one pixel of leading. Documents taller than 2016 pixels are split into pages at blank pixel rows, so no line is cut; all pages are sent in one request in reading order. The budget of Eq. (1) is enforced on the unsplit image; splitting can add one patch row per page, which made 10 of the 180 reallocated long-context images 1.2% more expensive than their uniform counterparts. All reported costs are billed tokens and include this effect.

Detector. After stripping the characters `.,;:()[]"'` from both ends, a word is flagged if it matches `[0-9a-f]{8,}`, `\d{5,}`, `/\S*\d\S*`, `\d+\.\d+\.\d+\.\d+`, `\S+/\S+\.\w{1,5}`, or `sha256=`, or if it contains at least one letter and one digit and either contains `[A-Za-z0-9_\-]{20,}` or is at least 16 characters long with a 16-character window whose character entropy is at least 3.5 bits.

Baselines. U tries Menlo, Arial, and Arial Narrow at sizes 14 down to 5 px and keeps the largest size that fits the budget (ties broken by font order); at the 5 px and 6 px budgets it selects 6 px Arial and 8 px Arial Narrow for most documents. Z renders at 10 px and shrinks the image by repeated factors of 0.98 with Lanczos resampling until it fits the budget (final scale 0.52–0.64). K and X append, after the question, the line "Exact strings copied verbatim from the text block in the image:" followed by one flagged string per line.

Prompts and scoring. RULER prompts follow the official template with the context replaced by "[The text is rendered in the image above.]" and ask for a JSON list of values; a gold value counts as recalled if it appears (case-insensitively) in any returned string. LongBench prompts are the official ones for each task; the first output line is scored with the official QA F1. For the plain-text arm the context is inserted verbatim into the same template.

Router threshold. τ is chosen per reader by two-fold cross-validation on the density sweep (folds by item parity), maximizing recall minus 0.5 times the cost relative to plain text over $\tau \in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.75, 1, 1.5, 2, 4, \infty\}$. Selected values were 0.1–0.5 with median 0.3, which we use for all other experiments.

B. Per-Reader Results

Tables 5 and 6 give per-reader numbers for Figs. 3 and 4.

Table 5. Per-reader recall for the length study (Fig. 3).

	32K		64K		128K	
Reader	A	S	A	S	A	S
Claude Sonnet 5.5	0.57	**0.86**	0.51	**0.96**	0.45	**0.97**
Gemini 3.5 Flash-Lite	0.47	**0.91**	0.26	**0.74**	0.21	**0.70**
GPT-6 Luna	0.09	**0.59**	0.12	**0.55**	0.09	**0.50**
Qwen3-VL-235B	0.02	**0.60**	0.02	**0.55**	0.06	**0.54**
Llama 4 Maverick	0.01	**0.50**	0.00	**0.54**	0.00	**0.47**
MiniMax-M3	0.00	**0.40**	0.00	**0.49**	0.00	**0.56**
Mean	0.20	0.64	0.15	0.64	0.14	0.62

Table 6. Per-reader recall in the hazard-density sweep, K/X (anchors on uniform rendering / on our rendering).

Reader	k=1	4	16	64	128	256
Claude Sonnet 5.5	1.00/1.00	1.00/1.00	0.88/0.96	0.60/0.72	0.28/0.44	0.16/0.24
Gemini 3.5 Flash-Lite	0.96/1.00	0.96/1.00	0.68/0.88	0.36/0.28	0.28/0.16	0.08/0.08
GPT-6 Luna	0.96/1.00	0.88/1.00	0.60/0.92	0.32/0.32	0.20/0.12	0.12/0.12
Qwen3-VL-235B	1.00/1.00	1.00/1.00	1.00/1.00	0.04/0.04	0.00/0.00	0.00/0.00
Llama 4 Maverick	1.00/1.00	0.80/0.96	0.48/0.44	0.48/0.20	0.16/0.16	0.08/0.04
MiniMax-M3	1.00/1.00	1.00/0.96	0.72/0.76	0.52/0.76	0.48/0.52	0.00/0.12

C. Short Documents: Mechanism Study

Before the long-context experiments, we studied 1.6K-character documents that fit one small image, with six identifiers and three one-word prose facts per document ("The release codename is . . . "), both on synthetic text and on real package manifests whose lines contain file paths and 43-character base64 hashes. This setting isolates the allocation mechanism but is not a deployment target: for such short inputs an image is often not cheaper than the text.

Reallocation helps every reader; prose can pay for it. Table 7 reports word-wrapped reallocation against uniform rendering. Identifier exact-match improves significantly in all 24 reader–condition cells. Prose probes are unaffected on synthetic text, but on real manifests some weaker readers lose prose accuracy (Llama up to -0.39), because whole manifest lines are enlarged and prose shrinks from 6 to 5 px. On long documents, where hazards occupy a small fraction of the text, this cost does not appear (Tab. 3).

The gain depends on density and reader. Figure 5 sweeps four densities from 7.8 to 21.1 characters per vision token on synthetic documents. The gain follows an inverted U whose peak moves to higher densities for stronger readers, as discussed in Sec. 5.6.

Table 7. Short documents (≈1.6K characters, one image, N=30). Difference between word-wrapped reallocation and uniform rendering in identifier exact-match (ids) and in accuracy on three prose probes (prose); column headers give data and characters per vision token. */†: 95% CI above/below zero.

	synthetic, 21		synthetic, 14		manifests, 14 (a)		manifests, 14 (b)	
Reader	ids	prose	ids	prose	ids	prose	ids	prose
Claude Sonnet 5.5	+0.35*	+0.07	+0.19*	+0.06	+0.21*	-0.10	+0.13*	-0.08
Gemini 3.5 Flash-Lite	+0.33*	+0.01	+0.04*	-0.01	+0.23*	+0.00	+0.19*	-0.01
GPT-6 Luna	+0.21*	+0.06	+0.41*	+0.03	+0.17*	-0.16†	+0.18*	+0.03
Qwen3-VL-235B	+0.04*	+0.01	+0.44*	-0.08	+0.06*	-0.04	+0.08*	-0.00
Llama 4 Maverick	+0.05*	+0.00	+0.66*	-0.39†	+0.18*	-0.38†	+0.27*	-0.38†
MiniMax-M3	+0.28*	-0.03	+0.57*	-0.02	+0.17*	-0.21†	+0.21*	-0.14†

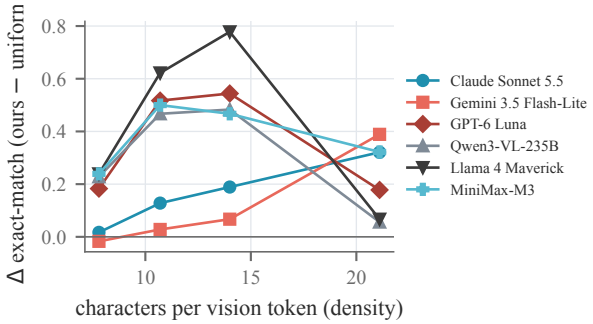


Figure 5. Gain in identifier exact-match of reallocation over uniform rendering versus rendering density (short synthetic documents, N=30 per point).

Readers do not know when they misread. Allowing an "[unreadable]" answer did not reduce silent errors: MiniMax-M3 used it for at most 6% of identifiers, and its rate of wrong-but-well-formed answers did not decrease. This supports preventing errors at render time rather than relying on the reader's self-assessment.

Agent-based readers can leak across items. In a pilot, Claude readers accessed through an agent interface read several images in one session. Among their wrong prose answers, 50–100% were names that appeared in *other* images of the same session, versus 0–10% for a stateless API reader. All agent-based readings reported in this paper therefore use a Latin-square design in which each agent sees each item in only one form (Sec. G).

D. Negative Results

Global detector. A detector that also flagged four-digit numbers and long letter-only tokens (later replaced by the one in Sec. 3.2) flagged on average 85 and 112 words per MultiFieldQA and HotpotQA document, 62% of them years. On one LongBench sample it lowered F1 by 0.05 on average, with 3 of 12 cells significantly negative; on a second sample it was neutral (−0.005, 1 of 12 negative). With the final detector the difference to uniform rendering is +0.000 (0 of 12 negative) on the second sample.

Word wrapping. Wrapping prose at word boundaries wastes part of every line; on long documents at the 5 px budget this left no room to enlarge anything, so that variant matched or exceeded the uniform cost while gaining little (+0.00 to +0.12 recall). Character wrapping for prose (Sec. 3.3) removes this waste.

Local instead of global shrinking. Shrinking only the prose within K words of a flagged string, with the smallest feasible K, was harmless on LongBench (+0.003, 0 of 12 negative) but retained only 84% of the RULER gain at the 6 px budget, because it fills the whole budget with slightly smaller enlarged strings. It brought no benefit over global shrinking.

Other variants. On short manifests, enlarging only the hash and path substrings of a manifest line freed almost no space. Adding one pixel of line spacing to prose-only lines did not change prose accuracy (+0.00, 95% CI $[-0.09, 0.09]$). Keeping prose at the uniform size left room only for 6–8 px identifiers and recovered 13–24 points less than full reallocation.

E. Natural-Identifier Questions

For each of six LongBench tasks we shuffled the documents of at most 40K characters with a fixed seed and kept the

first ten that contain a usable target. A target is a word flagged by the detector of Sec. 3.2 that, after stripping surrounding punctuation, has 6–64 ASCII characters including a digit, starts with a letter or digit, and is preceded by six words none of which is flagged; the six-word phrase must occur exactly once in the document. We exclude dataset artifacts: Qasper reference placeholders (e.g., `BIBREF`), formula fragments, characters replaced by "?" during ASCII normalization, and words glued across a missing space (a period, parenthesis, or comma directly followed by a letter). A first version of the set without the last two filters contained such artifacts in 14 of its 60 questions; because anchors and enlarged rendering present the glued string as a single unit, that version favored our arms, and we discarded it. The prompt asks for the exact string that follows the phrase; scoring is the same recall as for RULER.

F. Real-Corpus Profile

Table 8 lists the routing profile of real corpora: 13 English LongBench tasks (20 chunks each), Paul Graham essays, Python source files from an installed package environment, local system logs, and Python package `RECORD` manifests (36–40 chunks each). Every chunk is rendered and reallocated so that r uses the actual image cost.

Table 8. Routing profile of real corpora (32K-character chunks, τ=0.3). Flags per 1K characters, median and 90th-percentile routing ratio r, share routed to image+anchors (X), and estimated cost relative to plain text. Text tokens are estimated with a linear fit to billed text-only reads (0.208 tokens per ordinary and 0.737 per flagged character; mean absolute relative error 1.1%).

Corpus	n	flags/1K	r med	r p90	→X	cost/T
LB/narrativeqa	20	0.06	0.00	0.01	95%	0.27
LB/qasper	20	0.18	0.03	0.13	100%	0.19
LB/multifieldqa_en	20	0.85	0.05	0.98	75%	0.43
LB/hotpotqa	20	0.17	0.01	0.11	95%	0.25
LB/2wikimqa	20	0.06	0.01	0.05	100%	0.21
LB/musique	20	0.06	0.00	0.02	100%	0.22
LB/gov_report	20	0.04	0.00	0.02	100%	0.21
LB/qmsum	20	0.00	0.00	0.00	100%	0.23
LB/multi_news	20	0.06	0.00	0.05	100%	0.21
LB/triviaqa	20	0.23	0.08	0.19	95%	0.23
LB/passage_retrieval_en	20	0.04	0.00	0.02	100%	0.21
LB/lcc	20	1.40	0.40	0.83	25%	0.81
LB/repobench-p	20	1.43	0.52	0.87	5%	0.96
pg_essays	36	0.01	0.00	0.00	100%	0.22
python_source	40	2.62	0.67	1.47	12%	0.91
system_logs	40	11.84	3.73	4.09	0%	1.00
pkg_manifests	40	11.32	16.09	16.72	0%	1.00

G. Additional Claude Reader

We also used Claude through an agent interface as a reader that is never used to choose any setting. For $k \in \{1, 4, 16, 64, 256\}$, ten items of the density sweep were read as X, K, and plain text in a Latin square: 15 agents each read ten different items, each in one form, using only file-reading tools. Costs are estimated with Eq. (1) for images and with the linear text-token fit of Tab. 8. Table 9 shows that this stronger reader tolerates higher density than the API readers (0.90 for X at k=64, against 0.60 for K), and with τ=0.3 the routed system reaches 0.98 recall at an estimated 0.61× the plain-text cost. Agents may resample images when reading them, and N=10 per cell is small, so these numbers are indicative.

Table 9. Additional Claude reader (agent interface, one item per form per agent; N=10 per cell). Recall and estimated cost relative to plain text.

k	X	K	T	cost X/T	cost K/T
1	1.00	1.00	1.00	0.23	0.23
4	1.00	1.00	1.00	0.17	0.24
16	0.90	1.00	1.00	0.22	0.27
64	0.90	0.60	1.00	0.36	0.36
256	0.20	0.00	1.00	0.59	0.60

H. Reproducibility and Ethics

Code and data. The renderer, detector, router, task generators, reader clients, analysis scripts, all logged model outputs, and the scripts that produce every figure and table will be released. Task data are generated from public sources (RULER templates with Paul Graham essays, LongBench) with fixed seeds.

Compute and cost. No model was trained. All readers were accessed through provider APIs between November 1 and 3, 2026; reader versions are identified by their provider model names in Sec. 4. Pay-per-token API charges for all experiments, including pilots, were about $51; MiniMax-M3 and the agent-based Claude reader were used under subscription plans.

Ethics. The study involves no human subjects and no personal data. The system logs profiled in Sec. F were processed locally to count flagged strings; their contents are not released. Better verbatim fidelity under compression could in principle help extract identifiers from rendered documents, but the method only changes how a document the user already holds is presented to a model.